

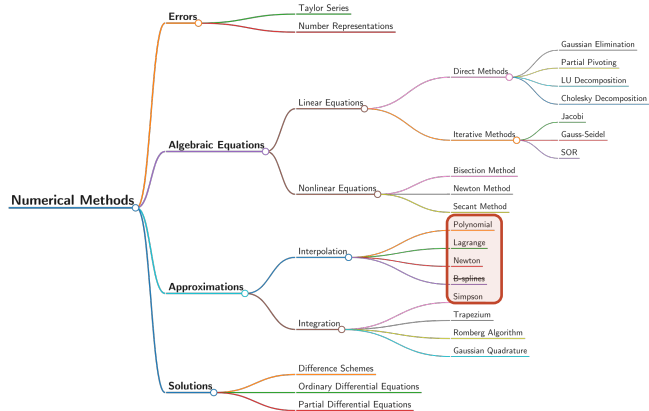
Polynomial Interpolation

CTMS-MAT-13 • Interpolation Methods

Dr. David Sinden

September 10, 2026

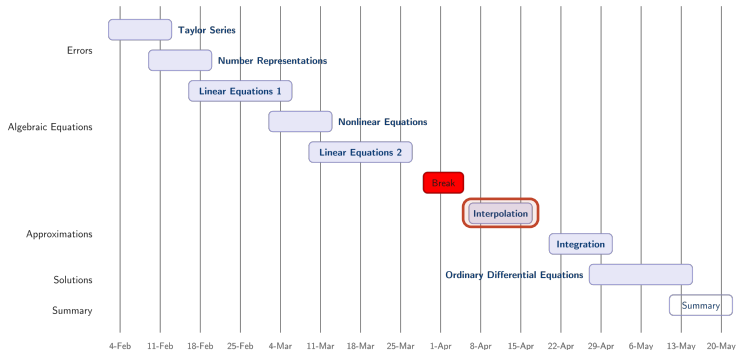
Where this sits in the course



Full outline: djps.github.io/docs/numericalmethods/intro/intro/

Schedule

Schedule 2026



Notes: djps.github.io/teaching/numerical_methods.pdf

Today

The problem

Monomial basis

Lagrange interpolation

Aitken's algorithm

Newton's basis

The problem

One-dimensional curves, high-degree polynomials

The underlying idea is **orthogonality**, in a wider sense than usual:

- in $\mathbb{R}^n$, $\vec{a} \cdot \vec{b} = 0 \iff \langle a, b \rangle = 0$;
- for functions, with $\langle a, b \rangle = \int a(x) b(x) dx$, the same statement defines orthogonal *polynomials*.

A basis is what turns a curve-fitting question into a linear-algebra question. Choosing it well is the whole lecture.

Aside: F. Chollet, *Deep Learning with Python*, 2nd ed., Chapter 6 — on the claim that machine learning is “just interpolation”.

Setup

Given **values** $p_0, \dots, p_n \in \mathbb{R}$ at **nodes** $u_0, \dots, u_n$, a function p with

$$p(u_i) = p_i, \quad i = 0, \dots, n,$$

is an **interpolant**.

Goal: for a chosen set of basis polynomials $\{\varphi_i\}$, find coefficients $\alpha_0, \dots, \alpha_n$ such that

$$p(u) = \sum_{i=0}^n \alpha_i \varphi_i(u)$$

interpolates the data. Note $n + 1$ conditions and $n + 1$ unknowns.

The collocation system

The $n + 1$ interpolation conditions are linear in the α_i :

$$\sum_{i=0}^n \alpha_i \varphi_i(u_j) = p_j, \quad j = 0, \dots, n,$$

that is $\bar{\Phi} \vec{\alpha} = \vec{p}$ with the **collocation matrix** $\bar{\Phi}_{ji} = \varphi_i(u_j)$.

TAKEAWAY

The interpolant is unique; only the *matrix* depends on the basis. Pick a basis that makes $\bar{\Phi}$ easy to solve — diagonal, triangular, anything but dense.

Monomial basis

$\varphi_i(x) = x^i$ and the Vandermonde matrix

With $\varphi_0 = 1$, $\varphi_1 = x$, $\varphi_2 = x^2$, ... the case $n = 2$ gives

$$p(u) = \alpha_2 u^2 + \alpha_1 u + \alpha_0,$$

every quadratic. For $\bar{\Phi}$ to be invertible the φ_i must be linearly independent — a genuine basis.

With nodes $u_i = i + 1$ the collocation matrix is the **Vandermonde matrix**

$$\bar{\Phi} = \begin{pmatrix} 1 & 1 & \cdots & 1^n \\ 1 & 2 & \cdots & 2^n \\ \vdots & & & \vdots \\ 1 & (n+1) & \cdots & (n+1)^n \end{pmatrix}.$$

Its columns become nearly parallel as n grows: **ill-conditioned**.

TAKEAWAY

The monomials are a basis, but a bad one — the natural choice is the numerically worst choice.

Monomial basis: example, $n = 2$

$$\begin{array}{c|ccc} u_i & 1 & 2 & 3 \\ \hline p_i & 2 & 5 & 10 \end{array} \quad \Rightarrow \quad \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 5 \\ 10 \end{pmatrix}.$$

Solving gives $\vec{\alpha} = (1, 0, 1)^\top$, i.e.

$$p(u) = u^2 + 1.$$

A full 3×3 solve for three data points. For $n = 20$ this is both expensive and inaccurate.

Lagrange interpolation

Lagrange: choose the basis to fit the data

Write the interpolant as

$$p(u) = \sum_{i=0}^n p_i L_i^n(u),$$

where n is an *index*, not a power, and the **Lagrange polynomials** are defined by the Kronecker-delta property

$$L_i^n(u_j) = \delta_{ij} = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

The coefficients *are* the data. The collocation matrix is the identity: no system to solve.

Explicit formula

$$L_i^n(u) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{u - u_j}{u_i - u_j}$$

— each factor vanishes at one other node, and the denominator normalises

$$L_i^n(u_i) = 1.$$

For $n = 2$:

$$L_0^2(u) = \frac{(u - u_1)(u - u_2)}{(u_0 - u_1)(u_0 - u_2)},$$

$$L_1^2(u) = \frac{(u - u_0)(u - u_2)}{(u_1 - u_0)(u_1 - u_2)},$$

$$L_2^2(u) = \frac{(u - u_0)(u - u_1)}{(u_2 - u_0)(u_2 - u_1)}.$$

Check: $L_0^2(u_0) = 1$ and
 $L_0^2(u_1) = L_0^2(u_2) = 0$.

They also satisfy the
partition of unity

$$\sum_{i=0}^n L_i^n(u) = 1,$$

Aitken's algorithm

Recursive Lagrange polynomials

- $n = 0$: $L_0^0(u) = 1$;
- $n > 0$: $L_i^n(u) = L_i^{n-1}(u) \frac{u - u_n}{u_i - u_n}, \quad i = 0, \dots, n-1$;
- and by partition of unity, $L_n^n(u) = 1 - \sum_{i=0}^{n-1} L_i^n(u)$.

Substituting the second into the third:

$$L_n^n(u) = \sum_{i=0}^{n-1} L_i^{n-1}(u) - \sum_{i=0}^{n-1} L_i^{n-1}(u) \frac{u - u_n}{u_i - u_n} = \sum_{i=0}^{n-1} L_i^{n-1}(u) \left(1 - \frac{u - u_n}{u_i - u_n} \right).$$

TAKEAWAY

Adding a data point does not require regenerating the basis — degree n is built from degree $n-1$.

Aitken's algorithm

Start from the data, $p_i^0 = p_i$. Substituting the recursion into $p(u) = \sum_i p_i^0 L_i^n(u)$ and collecting terms gives, with

$$p_i^1 = p_i^0 \frac{u - u_n}{u_i - u_n} + p_n^0 \left(1 - \frac{u - u_n}{u_i - u_n} \right), \quad i = 0, \dots, n-1,$$

the *same* interpolant one degree lower:

$$p(u) = \sum_{i=0}^{n-1} p_i^1 L_i^{n-1}(u).$$

Repeating,

$$p_i^{k+1} = p_i^k \frac{u - u_{n-k}}{u_i - u_{n-k}} + p_{n-k}^k \left(1 - \frac{u - u_{n-k}}{u_i - u_{n-k}} \right).$$

The tableau

$$\begin{array}{ccccccc} p_0^0 & p_0^1 & p_0^2 & \cdots & p_0^n & & \\ p_1^0 & p_1^1 & p_1^2 & & & & \\ \vdots & \vdots & & \ddots & & & \\ p_{n-1}^0 & p_{n-1}^1 & & & & & \\ p_n^0 & & & & & & \end{array}$$

Each entry is a convex-style combination of the one to its left and the one below-left:

$$\longrightarrow \times \frac{u - u_{n-k}}{u_i - u_{n-k}},$$

$$\searrow \times \left(1 - \frac{u - u_{n-k}}{u_i - u_{n-k}} \right).$$

The answer is p_0^n .

Complexity $\mathcal{O}(n^2)$ — but roughly half the work of forming $p(u)$ explicitly, and it evaluates at a *given* u without ever constructing the polynomials.

Worked example

$$\begin{array}{c|ccc} u & 0 & 1 & 2 \\ \hline p & 2 & 4 & 3 \end{array} \quad \text{evaluate at } u = \frac{1}{2}.$$

Step 1 ($k = 0$, combining with $u_2 = 2$):

$$p_0^1 = 2 \cdot \frac{3}{4} + 3 \cdot \frac{1}{4} = \frac{9}{4}, \quad \text{since } \frac{1/2-2}{0-2} = \frac{3}{4},$$

$$p_1^1 = 4 \cdot \frac{3}{2} + 3 \cdot \left(-\frac{1}{2}\right) = \frac{9}{2}, \quad \text{since } \frac{1/2-2}{1-2} = \frac{3}{2}.$$

Step 2 ($k = 1$, combining with $u_1 = 1$): $\frac{1/2-1}{0-1} = \frac{1}{2}$, so

$$p_0^2 = \frac{9}{4} \cdot \frac{1}{2} + \frac{9}{2} \cdot \frac{1}{2} = \frac{9}{8} + \frac{9}{4} = \frac{27}{8} = 3.375.$$

Newton's basis

Newton's basis

Take basis polynomials whose roots are the earlier nodes:

$$\varphi_0(u) = 1,$$

$$\varphi_i(u) = (u - u_0)(u - u_1) \cdots (u - u_{i-1}), \quad i = 1, \dots, n.$$

Then $\varphi_i(u_j) = 0$ whenever $j < i$, so the collocation matrix is **lower triangular** — solved by forward substitution, no inversion.

TAKEAWAY

Three bases, three matrices: dense and ill-conditioned (monomial), the identity (Lagrange), lower triangular (Newton). Same interpolant every time.

The interpolant is unique.
Only your basis is a choice.

Summary

- Interpolation through $n + 1$ points is a linear system $\bar{\Phi}\vec{\alpha} = \vec{p}$ in the coefficients of a chosen basis.
- The monomial basis gives the Vandermonde matrix: dense, and ill-conditioned for large n .
- Lagrange polynomials satisfy $L_i^n(u_j) = \delta_{ij}$, so the coefficients are the data and no system is solved.
- Partition of unity gives a recursion in the degree, and hence Aitken's algorithm: evaluate $p(u)$ at one point in $\mathcal{O}(n^2)$ without building the basis.
- Newton's basis makes $\bar{\Phi}$ lower triangular — new data appended without redoing the old work.

Take-home messages

1. **Choose the basis for the matrix it produces.** Conditioning, not elegance, decides.
2. **Evaluate, don't construct.** If you need p at a few points, Aitken is cheaper than forming the polynomial.
3. **High degree is not high accuracy.** Many nodes make the interpolant oscillate; prefer low-degree piecewise interpolation (splines) over one polynomial of degree 20.
4. **Interpolation is the backbone of quadrature.** Integrating an interpolant is exactly how the trapezoidal and Simpson rules arise.

Next: numerical differentiation and integration.